A Data-Analysis Agent, Built From Scratch

Part 1: two benchmark papers on autonomous scientific analysis
Part 2: a design built against what they found

Musel Tabares Pardo

July 2026

Part I

The papers

Motivation

- AI agents have improved fast — they now **write and run code in a loop**, unsupervised.
- So we want to **assess how well they do real scientific research**.
- To know, we need **benchmarks that give us information about that**.

Two papers, two benchmarks — each evaluates a *model* + *a harness* in a container

- **GeneBench-Pro** (OpenAI) — **129 problems**, genomics. **Deterministic Python grading.**
- **DrugDiscoveryBench** (Scale AI & Phylo) — **82 tasks**, drug discovery. **LLM-judge grading.**

Outline

GeneBench-Pro — the motivation

A gap in biology benchmarks

Real scientific analysis is **far more complex** than what the benchmarks measure, and there are **too few representative** ones. GBP exists to close that gap.

Pointed at the right bottleneck

The limiting factor is **no longer sample collection** but **downstream computation and analysis**. So a good benchmark assesses the step **AI agents could most help with**.

"downstream computation and analysis, rather than data generation, have become the central scientific bottleneck." — GBP, p. 2

GBP — characteristics

- **129 problems**, 10 domains. **Fully simulated** — causal structure known, nothing memorisable.
- Docker, **no internet**, a minimal prompt. Files are **deliberately messy** (sentinels, batch effects, missingness).

The unit to remember — the *decision point*

A **fork where two reasonable paths give *different numbers*** — e.g. a `-999` column: take its mean as-is, or recognise it as *missing* and drop it first. **3–13 per problem** (median 6), chained.

GBP grades **only the final number** — but a problem is built so the right number is reachable **only by the right choice at every fork.** So one number tests judgement, not just arithmetic.

GBP — why build it synthetically

Two ways such benchmarks break

Messy historical data → many defensible paths → the score measures the **author's preference**, not skill.

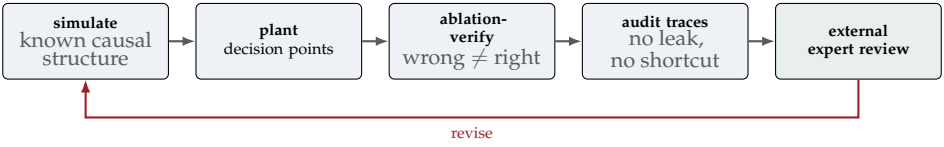
Numerically insensitive → a **fundamental error** still passes.

GBP's fix: *simulate*

They know the true data-generating process — so they can **compute** the answer (not assume it), **prove by ablation** the naive path lands on a different number, and be sure **nothing was memorised.**

With real data you can only *assume* a path is wrong. Simulated, you can *prove* it.

GBP — construction & validation



External review — the human check on machine-made tasks

Synthetic data risks being **unrealistic** or **under-specified**. So **11 domain experts** (grad students to professors) reviewed 84 problems; **82 kept**. Each asked: **realistic? is the answer *uniquely* recoverable from the given data? are the methods appropriate?** Feedback drove redesigns.

GBP — grading

Grade against the **recoverable** estimate — *not* the hidden generating parameter:

Naive (broken)

True knob $\beta = \mathbf{0.30}$; but a flawless analyst recovers $\hat{\beta} = \mathbf{0.283}$ from the finite files. Grade against 0.30: **a perfect analysis is marked WRONG.**

GBP (right)

Grade against **0.283** — what a correct analysis can actually recover. **Tests the analysis, not the dice.**

Why they differ: the staged file is a **finite sample**, so noise moves the recoverable value off the knob — like getting 6 heads in 10 flips of a fair coin.

Binary, deterministic Python — no LLM judge. Partial credit measures effort; binary measures usefulness.

GBP — the results are brutal

Configuration	Pass rate	Never solved (0/all tries)
GPT-5.6 Sol Pro (Extended) — *best in paper*	**31.5%**	
GPT-5.6 Sol (max) — *best mainline*	**28.7%**	**45.7%**
Claude Opus 4.8 (max) — *strongest non-GPT*	16.0%	
Non-GPT range: **0.6%** (MiniMax M2.7) to 16.0% (Opus 4.8)		

28.7% is an average — read the last column

The best mainline model scores **zero on 45.7% of problems**, across all ten tries. Effort helps then saturates: none→max is 3.7 → 28.7, but high→max is only +1.9.

GBP — limitations

- **OpenAI evaluates its own models.** The leaderboard is topped by GPT-5.6, same lab — the headline-model choice and framing are in-house.
- **No human baseline.** 28.7% has **no ceiling to read against** — is an expert 90%? 60%? The paper does not say.

So the numbers say how good these models are at *these tasks* — not how far from a scientist.

Outline

DrugDiscoveryBench — the motivation

The bigger goal: agents that can do science — form hypotheses, plan experiments, use domain tools, read the results.

Why *drug discovery*, and why the *early* part?

Early drug discovery is a **narrow but especially testable** slice of that goal: it turns a hypothesis into **evidence-gathering and analysis** over **structures, assays, patents, papers, and biological databases** — each step **grounded in a real source** and ending in a **single answer you can check**.

DDB — what it measures

*A drug is a small molecule that jams a **target** protein — a key in a lock. DDB tests the early, computational half of finding one.*

The drug-discovery funnel

target ID → **hit ID** → **hit-to-lead** → **lead opt.** | candidate selection → preclinical → clinical

find the protein → **find molecules that stick** → **keep the best** → **tune them**. DDB grades these four and **stops at candidate selection**.

Each task is a *chain*: retrieve → parse → filter → compute → one answer

The steps **feed each other**, so **one slip early poisons everything after it.** Just like GBP's cascading decision points — which is why grading is all-or-nothing.

DDB — the tasks: real artifacts, one verifiable answer

- **82 tasks**, written *and* graded by domain experts.
- Each tied to **one real artifact** (a PDB structure, a patent, a paper, a database record) that **already contains the answer.**
- Tagged by **skill** (7) × **funnel stage** (4) — hit-to-lead densest (34 tasks), lead-opt thinnest (10).

GBP simulates the truth; DDB looks it up.

One task, start to finish (lead optimisation)

Compound B jams KRAS harder than A — why? Two **co-crystals** (the protein frozen with each drug inside), `6USX`, `6UT0`:

overlay → find the **residue** (amino-acid bead) that **moves** and touches B. **Answer: `THR58`.**

Answers stay small: a count, a float, a molecule as text (SMILES — e.g. ethanol = `CCO`), a ranking.

DDB — the workbench (Scale-Biomni)

*The agent is a **general coding agent** — no drug-discovery training. DDB just hands it a biology toolbox.*

What the agent is given

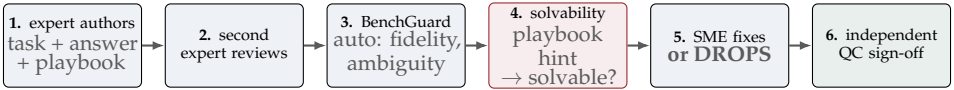
- **226 functions / 22 domains, imported like any Python library** (`query_uniprot(...)`) — no special wrapper between model and tools.
- **76-file data lake** (reference tables on disk) + **117 packages**. **Internet on. 120 min/task.**

Sealed vs open — the contrast that will matter

GBP: no internet. **DDB:** internet + a 226-function toolbox.

The interface is general; the work is biomedical. And "open" means the agent can accidentally **look up the answer** (last limitation).

DDB — construction: the authoring pipeline



Gate 4 is the *entry ticket* — remember it for the results

A task is kept only if an agent **can solve it once handed the expert's playbook** (the worked recipe); the rest are **dropped**. Like a textbook that only keeps problems solvable *with* the worked solution — so later, "give them the playbook and they pass" is partly baked in.

DDB — how an open answer gets graded

*You can't == a paragraph. Each task ships with a **checklist**; an LLM ticks the boxes.*

A *weighted rubric* — small checks, two kinds (KRAS task)

outcome — scored: "answer is `THR58`" **process** — recorded, not scored: "fetched `6USX`+`6UT0`"

Pass = 100% of outcome checks. A half-right answer, a timeout, or a refusal all score 0.

One **LLM judge** (GPT-5.4) ticks the boxes — and it's *strict*

Concrete checks, so it's **matching a claim to a key**, not essay-grading. (Sanity check: two *other* LLMs re-graded a sample and agreed, $\kappa = 1.0$ — not a per-task 3-judge vote.) It forgives **last-digit rounding** on floats — but **IDs, accession numbers, counts and categories must match *exactly*.**

DDB — results

Setting	Harness	Pass
GPT-5.5 (xhigh) — *best*	mini-SWE-agent	**51.6%**
Gemini 3.5 Flash (high)	Gemini CLI	50.0%
Claude Opus 4.8 (max)	Claude Code	45.1%
GPT-5.5 (xhigh)	Codex *(native)*	43.9%
MiniMax M3 (xhigh) — *bottom*	mini-SWE-agent	**23.2%**

- **Even the best solves just over half** (51.6%) — with tools, internet and 2 hours.
- **Think longer → score higher** (effort low→ceiling: GPT-5.5 Codex 27.6 → 39.8 → 43.9). One model generation ≈ 8–20 pts.
- **The *harness* matters as much as the model** — swap GPT-5.5's wrapper, 43.9 → 51.6. "Same chef, different kitchen."

Scores average **3 runs** per setting (not best-of-n). Coverage differs too: Opus refuses 5–6 published-data tasks per run.

DDB — is the gap *planning* or *execution*?

Why do agents fail? Either they can't do the steps, or they can't figure out which steps. This experiment tells them apart.

Hand over the *plan*, never the answer

Give the agent the **playbook** (the recipe: which steps, which tools) and see whether a failing run is rescued.

Best *single* agent, unaided	51.6%
Any run solves it, pooling all agents (an *oracle* — nobody does this alone)	76 / 82
…then hand the ∼6 leftovers the plan	**80 / 82**

The conclusion

Give the **plan** and the stragglers fall ⇒ agents **can do the steps**; they just can't **choose them**.

The gap is unguided *planning*, not execution.

And when they fail, it's usually **dropping a constraint mid-run** (e.g. forgetting to filter for the disease) or picking the **wrong data representation** — exactly Part 1's Failure 2.

DDB — limitations

- **The judge was never checked against a human.** Every score is one LLM's (GPT-5.4), "validated" only against *two other LLMs*. Three LLMs agreeing may just mean they **share a blind spot** — and the judge is GPT, same family as the winner.
- **The internet leaked.** With web search on, an agent **found DDB's own results online and returned the answer without doing the analysis** (paper, p. 10 + Table 6). So some scores may be **lookups, not skill** — and only the one obvious case was caught.

Outline

Over half of failures are not coding failures

Failure mode (DDB, 226 failing runs)	**Share**
Domain reasoning — wrong premise, though inputs and tools are correct	**54.0%**
Derivation error — right approach, wrong calculation	18.6%
Retrieval — wrong source, or never reads a file	16.4%
Constraint — violates an explicit instruction	7.5%
Final-answer slip — every step right, reports it wrong	3.5%

The tools worked. The Python ran. The data was there.

Over half of failures are not coding failures. The next four slides make each concrete — and name the Part 2 mechanism it earns.

Failure 1 — it notices, and does nothing about it

> *"models often complete substantial portions of the workflow but exhibit a consistent gap between **noticing** and **acting**…"*

GeneBench-Pro, abstract, p. 1

The model runs `df.describe()`, sees the sentinel values, may even comment on them — **then cleans the column and runs the analysis it had already decided on.**

The observation never reaches the decision.

→ **Part 2:** a **Findings Ledger** — a noticed problem becomes an *open obligation*, and the exit is blocked until it is discharged.

Failure 2 — the question quietly degrades

DDB — the melanoma task

Mid-trajectory the qualifier *"melanoma"* silently drops. The ranking becomes a gene-wide count; **PTEN** wins on volume. (Truth: CDKN2A.)

GBP — a denominator trap

The estimand is over **the whole roster**; the agent standardises over **tested partners only** — a plausible, wrong answer.

A constraint or population **silently drops**, and every step after is a correct analysis *of the wrong question*.

→ **Part 2:** a **Question Contract** — write the estimand, population, units and constraints as state, and *re-show it every turn*, so decay becomes visible.

Failure 3 — its own code printed the right number

> *"its own code printed the correct group-level count of 1, but the final tally used the atom-level count of 2… It reported 8 interactions instead of 7."*

DrugDiscoveryBench, p. 27

The analysis, the code, and the printed output were all **right**. **The number in the answer was not the number in the output.**

→ **Part 2: Evidence Grounding** — regex every reported number against real stdout; reject the answer if a number was never printed. *Fifteen lines.*

Failure 4 — nobody checks the answer at the end

"The last chance to catch the slip is at the final answer… None of the failing models caught this."

DrugDiscoveryBench, p. 13

Claude Opus answers **BRCA2** — a *breast*-cancer gene — to a **melanoma** question. Any expert would stop dead. **The agent submits it.**

One diagnosis, two labs: the bottleneck is **procedure**, not code, knowledge, or compute — **and procedure is something an engineer can build.**

→ **Part 2:** finishing is a **gated action** — an LLM judge must *approve* the draft before `submit_answer` returns.

Part II

The design

An LLM, a loop, and the gates on top

Outline

Part 2, Option A

A question in English + a folder of data files. Explore, choose an approach, write and run code, check intermediate results, return a **structured answer**.

The base model already writes competent pandas. The work is making it *keep the thread*.

Part 1 named the ways agents lose it. Each mechanism answers one — turn the **question, finding, number** into state the model cannot drop, then add a **detector** that feeds those and a **verifier** that checks the exit:

What gets dropped	The failure	The mechanism
the question	the melanoma qualifier evaporates	**Ledger 1 — Question Contract**
the finding	notices the sentinel, ignores it	**Ledger 2 — Findings Ledger**
the number	printed 1, reported 2	**Ledger 3 — Evidence Grounding**
never noticed	the weak model misses the smell	**Detector — deterministic turn-0 profiler**
nobody checks	submits an unchecked answer	**Verifier — fresh context, gated exit**

Spoiler from the evaluation: the **detector** — the humblest row — is the one that carries the result.

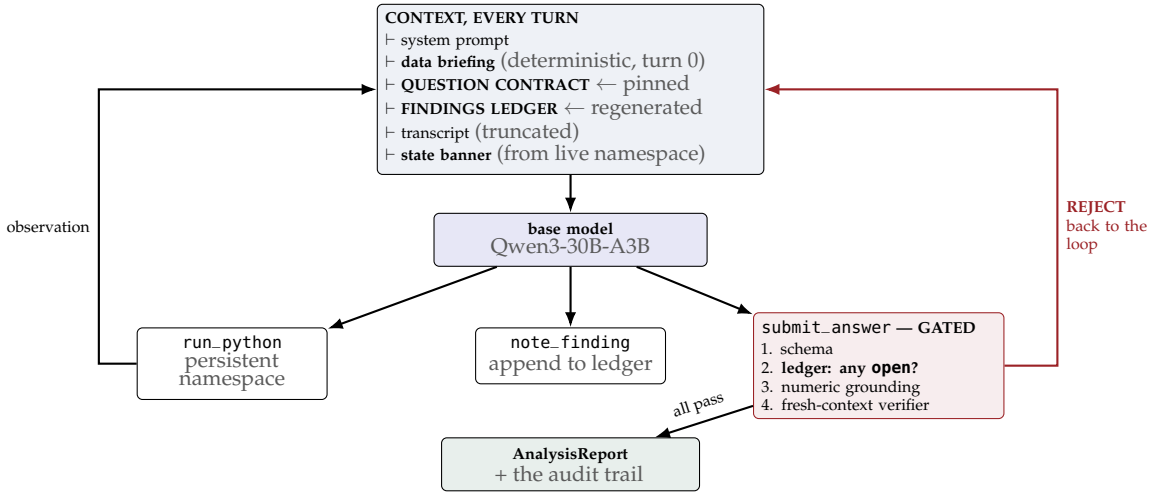
The base — what an agent actually is

1. **The LLM is a pure function.** `messages` → `message`. No memory, no state.
2. **Tool calling is not the model running code.** It emits a *request*; **your program** executes it and appends the result.
3. **The agent loop is a `while` loop.** (ReAct, Yao et al.)

```
while step < budget:
    msg = llm(transcript, tools)          # think
    if msg.tool_calls:
        result = execute(msg.tool_calls)  # act
        transcript.append(result)         # observe
    if submitted: break
```

That is the whole agent — ∼40 lines. Everything interesting surrounds the loop.

The architecture



The loop is unremarkable. **All of the design is the gated box on the right and the two ledgers pinned on top.**

Five tools — and why not more

- `run_python(code)` — a **persistent namespace**, like Jupyter cells. One general tool beats *N* specific ones: anything pandas can do, the agent can do.
- `set_contract(...)` — state the question as structured state (Ledger 1).
- `note_finding(...)` / `resolve_finding(...)` — open and close ledger obligations (Ledger 2).
- `submit_answer(...)` — the **gated** exit.

Only *one* tool does analysis; the other four are bookkeeping the model must keep. DDB gave its agents **226 functions** and still scored 51.6% — the models are **not failing for lack of affordances**.

The single most important context rule

The model never sees the data — only metadata, summaries, and the output of code it wrote.

Scale-invariant: a 200,000-row frame and a 200-row one cost the same tokens.

Addition 0 — the detector

*A frontier model usually notices the mechanical smells itself. A **weak** model does not — so the noticing has to be done for it, deterministically.*

A **deterministic profiler** runs at turn 0: shape, dtypes, nulls, cardinality — *and* checks for sentinels, duplicate ids, numeric-looking strings, group imbalance.

What it finds does **not stay a note** the model can scroll past — the ledger and gate make it an **obligation**.

Information can be ignored. An obligation cannot.

The division of labour the whole design rests on

deterministic code	finds the mechanical problems, and makes them **un-ignorable**
the model	decides what they **mean** for *this* question

It flags a *symptom* ("47 values of −999"), never a diagnosis or a fix — the analysis is still the model's to do. **That is what separates it from a worked playbook.**

Ledger 1 — the Question Contract

```python
class QuestionContract(BaseModel):
    estimand:   str   # the exact quantity
    population: str   # WHICH ROWS. denominator.
    units:      str   # percent? fraction? log?
    constraints: list[str]  # stated filters
    premises:    list[str]  # what it ASSUMES
    question_is_precise: bool   # REQUIRED
    ambiguities: list[str]
```

Example — a filled contract

`estimand`: mean biomarker change, wk 12
`population`: **arm A only**, completers
`premises`: [“arm A was randomised”]
`question_is_precise`: **true**

Two fields the evaluation demanded

`premises` — a question can be *wrong*. *“Which of the **four** sites…”* — there are **three**. Without it the agent answered anyway. **0/3 → 3/3.**

`question_is_precise` — a *required boolean*. With an optional `ambiguities` list, the agent left it empty **every time.**

A field the model *may* leave empty is a field it *will* leave empty.

Pinned every turn. Shown to the verifier at submit.

Ledger 2 — the Findings Ledger

```
note_finding(
  observation = "biomarker_x: 47 values "
                "of exactly -999.0",
  implication = "missing-data sentinels, "
                "not measurements",
  status      = "open",  # open|acted|dismissed
)
```

The rule that makes it bite

`submit_answer` is **hard-blocked** while *any* finding is `open`.

Close by **act** (point at the code step) or **dismiss** (say why it doesn't matter). The `implication` field is where the *propagation* happens — the step both papers watch it skip.

Example — a run's ledger

status	noticed → did
ACTED	88 `-999`s in a biomarker → excluded before any mean
ACTED	48 duplicate `patient_id`s → kept the last row
ACTED	`arm` imbalanced on `severity` → stratified
DISMISS	batch B on a shifted scale → never enters a *rate*

One is an **oracle**. The other is a **colleague.**

The rule, in the system prompt, as an absolute

Every number you report must have been printed by code you ran.

Enforced deterministically: regex the numbers, normalise (commas, %, 4 sig figs, $\times 100/\div 100$), require containment in the concatenated stdout. **No LLM.**

```
REJECTED: the value 0.62 in your evidence never appeared in any executed
output. Compute and print it before citing it.
```

An LLM reviewer *might* catch "printed 1, reported 2". A regex catches it every time, for free.

The gated exit

`submit_answer` runs four gates **in order — cheapest first**:

	Gate	Rejects when
1	**schema**	pydantic fails → the model self-repairs
2	**ledger**	*any* finding is still `open`
3	**grounding**	a reported number never appeared in real output
4	**verifier**	one LLM call — **only if 1–3 passed**

The verifier's one critical detail

Different model family, and **it does not see the agent's reasoning** — only the contract, the briefing, the code and its output, and the draft. Show it the chain of thought and it rubber-stamps. **The fresh context *is* the mechanism.**

Outline

The benchmark

How I built it — the simplest honest version

1. **Simulated two datasets** following GBP; **used a third, held-out** one.
2. Asked **Claude to author 28 tasks** — lookups, aggregations, and *planted traps*.
3. Grade with **binary, programmatic** metrics — ground truth is a pandas callable only the grader runs.

- `penguins.csv` — real, clean. Guardrails should buy *nothing*.
- `trial.csv` — synthetic clinical, known data-generating process.
- `sales.csv` — **a held-out domain**, e-commerce. **Never designed against.**

GBP's third principle, reused: *assert* the naive answer differs from the correct one by $\gg$ tolerance — it **caught two of my own broken tasks.**

The ablation — switch off one mechanism at a time

Remove this	Pass	Δ	Verdict
(nothing — the full agent)	**87%**	—	
the data briefing ← **the DETECTOR**, the turn-0 profiler	**60%**	−27%	**HURTS**
every guardrail at once	67%	−20%	**HURTS**
the Findings Ledger ← *centrepiece*	81%	−6%	**HURTS**
the fresh-context verifier	87%	−0%	no effect
the numeric grounding gate	87%	−0%	no effect
the Question Contract	88%	+1%	no effect

4,480 runs, 20/cell, ∼$16. Paired hierarchical bootstrap (GBP's scheme).

Twenty lines of pandas beat every gate combined

I built this around **gates**. The thing carrying it is the **detector** — the mechanism I spent the *least* time on.

The conclusion

A gate is only as good as the detector feeding it.

The gates weren't wrong — they were **redundant.** Remove the detector and they have nothing to gate.

Doesn't GBP say the opposite?

GBP: *frontier* models *consistently notice* — the headroom is in *acting*. **But that's a different model.** GBP runs GPT-5.6; my agent is **Qwen3-30B-A3B**, chosen because it is weak. For a frontier model, noticing is the solved half; for a 30B model, **noticing is not reliable** — so the profiler substitutes for it.

A hypothesis, not a result: the notice–act gap looks like a **frontier** problem; the *notice* gap like a **small-model** one.

I ran **one** 30B model — enough to *motivate* this boundary, not to prove it. It needs a sweep across model sizes I did not run.

The failure the average hid

	pass	landed on the naive answer
`t4_simpson` — Simpson's, **medical** (designed against)	40%	55%
`s4_simpson_sales` — Simpson's, **held-out**	35%	50%
`b3_ambiguous` — *"did the biomarker improve?"*	5%	—
25 easier tasks, 75–100%	carry the 87%	

On the confounded comparison, my agent is worse than a coin flip

It is **more likely to land on the naive answer (50–55%) than the right one (35–40%)**.

An average is where a failure goes to hide.

And there is **no detector for confounding** — so the roadmap is not more gates, it is **more detectors.**

The dashboard — and a live demo

`http://localhost:8501` `uv run streamlit run app.py`

Run the agent

- Ask a question, read the **audit trail**.
- **Toggle guardrails off and watch it fail.**

The evidence

- 4,480 runs, 8 ablations.
- The per-task table.

Caveat: the demo serves **Qwen3-4B** on a rented GPU — 7× smaller than the eval model, verifier not cross-family. **Live numbers are not comparable to the evaluation.**

Takeaways

1. Both papers agree, from opposite directions: agents fail by **losing the scientific thread** — not for lack of knowledge, tools, or compute.
2. So I spent the budget on **procedure**: turn the **question, finding, number** into state the agent cannot drop.
3. The evaluation said the **detector** matters most.

   A gate is only as good as the detector feeding it.
4. A boundary I can conjecture but not prove (one model, not a sweep): the notice–act gap looks **frontier**; the notice gap, **small-model** — and neither paper evaluates a small model.

Thank you.

Backup — per-task results (full agent, $n = 20$)

Task	Category	Pass	Naive
b3_ambiguous	ambiguous	**5%**	0%
s4_simpson_sales	trap:confounding	**35%**	**50%**
t4_simpson	trap:confounding	**40%**	**55%**
b2_false_premise_sex	false-premise	75%	0%
s3_refunds	trap:filter	75%	0%
t3_batch_units	trap:units	85%	10%
s13_ambiguous	ambiguous	85%	0%
s2_test_orders	trap:outliers	85%	0%
s10_holdout_rev_seg	trap:outliers	90%	0%
s11_holdout_conv	trap:constraint	90%	0%
s1_text_numbers	trap:dtype	90%	10%
s6_scope	trap:constraint	95%	0%
s9_false_premise	false-premise	95%	0%
h1_holdout_mean_severe	trap:sentinel	95%	0%

Task	Category	Pass	Naive
p1_lookup	lookup	100%	0%
p2_aggregate	aggregation	100%	0%
p3_groupby	groupby	100%	0%
p4_correlation	correlation	100%	0%
s5_age_sentinel	trap:sentinel	100%	0%
s7_groupby	groupby	100%	0%
s8_lookup	lookup	100%	0%
s12_holdout_age	trap:sentinel	100%	0%
t1_sentinel	trap:sentinel	100%	0%
t2_duplicates	trap:duplicates	100%	0%
t5_scope	trap:constraint	100%	0%
b1_false_premise_sites	false-premise	100%	0%
h2_holdout_rate	trap:duplicates	100%	0%
h3_holdout_scope	trap:constraint	100%	0%

25 tasks spanning 75–100% carry the 87%. The three red rows are the ones the papers were written about.

Backup — where the machinery works exactly as designed

`t3_batch_units` — assay batch B reporting 10× too large

0/20 without the guardrails $\longrightarrow$ **17/20** with them

The **detector** sees the shifted scale → it becomes an **open obligation** in the ledger → the **gate** blocks submission until it is discharged.

The whole thesis, in one task

Where a detector feeds it, the gate is worth 17/20. Where none does (confounding, ambiguity), it is worth nothing.

Backup — the instrument I built to stop fooling myself, lied

I ran **the identical benchmark twice**, either side of a change I had **proved was inert**:

Removing the Findings Ledger	Δ	**95% CI**
run A	-11.1%	$[-18.6, -4.3]$ — “significant”
run B	-1.8%	$[-8.9, +4.6]$ — “no effect”

With **10 runs/cell** the bootstrap resamples *from the ten outcomes I observed*: a `1/10` cell cannot reach the 60% the next run produced.

So I doubled to 20/cell — and report the *spread* between two halves

If a verdict flips between two runs of the same experiment, it was never a verdict. **It was weather.**